

Cache 与存储层次

怎样维护一个正确的高速副本

408 · 计算机组成原理 Topic CO-06

母问题：小而快的副本怎样证明“它就是那个数据”？

Cache 不是小型主存，而是带身份、有效性、修改状态和替换策略的副本系统：

Locality → Block Transfer → Placement → Tag / Valid → Offset → Replacement / Write State → AMAT

每次访问既要找到候选位置，又要证明候选行保存的是请求块，并决定副本修改何时向下传播。

本册定位与 Owner 边界

- **Owns:** 局部性、块/行、tag/index/offset、直接/组相联/全相联、hit/miss 状态机、3C、替换、写策略、容量位数、AMAT 与层次接口。
- **Uses:** CO-05 的存储粒度与延迟；CO-07 的地址翻译接口；CO-04 的 miss stall/重叠。
- **Stops:** OS Page Cache、文件缓存、页表/置换和总线 DMA 不在本册重新定义。

目录

1 为什么“按地址直接找数据”不够	3
2 地址三分与组织	3
3 Hit/Miss 生命周期	3
3.1 Read hit	3
3.2 Read miss	3
3.3 3C 模型	3
4 写策略是两条独立轴	4
5 替换与状态成本	4
6 贯穿母例：两个块反复驱逐	4
7 AMAT 与物理位数	4
8 地址翻译接口	4
9 五列概念边界	5

10 做题控制协议	5
11 全科坐标与跨册交接	5
12 考纲映射与来源处理	6

1 为什么“按地址直接找数据”不够

Cache 容量小于下一级，多个主存块必须复用有限行。因此访问必须依次回答：允许放在哪？当前行属于哪个块？该块是否有效？块内取哪一字节？若被修改，谁拥有最新副本？这五问生成 placement、tag、valid、offset、dirty 和 replacement。

局部性是收益来源而非正确性证明：时间局部性预测近期重访，空间局部性预测邻近访问；预测失败仍必须通过 tag/valid 维持正确性。

2 地址三分与组织

设地址宽度为 m 位，数据容量为 C 字节，块大小为 B 字节，相联度为 E ，则组数

$$S = \frac{C}{B \times E}, \quad b = \log_2 B, \quad s = \log_2 S, \quad t = m - s - b.$$

地址字段为 ‘tag | set index | block offset’。若按字编址或 B 用字表示，先统一单位。

组织	候选位置	优点	代价
直接映射	1 行	命中路径短、无需替换选择	冲突风险高
E 路组相联	同组 E 行	成本与冲突折中	E 个 tag 比较器和选择器
全相联	任意行	映射冲突最少	全部 tag 并行比较、替换成本高

3 Hit/Miss 生命周期

3.1 Read hit

索引定位候选组，比较 ‘valid=1’ 且 tag 匹配的行，再由 offset 选择块内字节/字。命中只证明副本身份和有效性，不表示下一级没有其他副本。

3.2 Read miss

choose invalid/victim → write back if dirty → fetch block → fill tag/data/state → retry request.

顺序流水线可能 stall；能否继续执行由 CO-04 和实现的非阻塞结构决定。

3.3 3C 模型

compulsory 是第一次访问；capacity 是理想全相联同容量也装不下；conflict 是有限映射额外造成的互逐。分类必须绑定块大小、容量、相联度和访问序列，不能把所有 miss 都叫 “Cache 太小”。

4 写策略是两条独立轴

Hit 时: write-through 同时更新下一级, 流量大但一致关系直接; write-back 只更新 Cache 并设 dirty, 逐出时才写回, 流量较小但状态复杂。Miss 时: write-allocate 先取块再写入; no-write-allocate 绕过 Cache 写下一级。write-back 不自动推出 write-allocate。

5 替换与状态成本

直接映射没有选择; 组相联/全相联需在候选中选 victim, 可用 FIFO、Random、近似/精确 LRU。精确 LRU 的元数据和更新逻辑随相联度增长, 不能笼统断言“每行只需 $\log_2 E$ 位”。增大相联度降低冲突, 可能增加 hit time、功耗和比较器数量。

6 贯穿母例：两个块反复驱逐

对直接映射 Cache, 将地址除以 B 得 block number, 再算 $set = block \bmod S$ 。若两个 block 的 set 相同而 tag 不同, 交替访问会互相驱逐, 即使其他组为空。提高相联度让它们在同组共存, 但需要额外比较与 victim 选择。

典型错误路径

只看 tag 相同就判 hit, 漏了 valid; 只提高 Cache 容量就声称冲突一定消失; 只看 hit rate 就宣称更快, 漏了更长 hit time 或更大 miss penalty。

7 AMAT 与物理位数

单级近似:

$$AMAT = T_{hit} + MR \times MP.$$

多级时 miss penalty 可能包含下一级 AMAT、写回脏块和总线占用。使用公式必须声明 hit time 是否包含并行 tag/data、MP 是额外时间、写回成本是否计入、stall 是否可重叠。

每行物理位数至少为 data、tag、valid, write-back 再加 dirty, 替换/ECC 元数据按题设另算:

$$C_{physical} = N_{lines}(data + tag + valid + dirty + metadata).$$

“Cache 容量”若只指数据区, 不能直接拿物理总位数替代。

8 地址翻译接口

PIPT、VIPT、VIVT 是 Cache 使用 VA/PA 与 TLB 的组织选择。VIPT 能并行的关键是索引位不依赖被翻译改变的 VPN 位; 具体约束由页内偏移、块大小和相联度推出, 不背固定上限。Cache hit 也不等于 TLB hit。

9 五列概念边界

概念 A	≠ 概念 B	真正区别与题面信号	混淆后果
Cache	≠ 主存	小容量带身份副本 vs 原始层级存储	把 tag/valid 当普通地址位
Tag	≠ Index	身份证明 vs 候选组定位	地址三分错位
Valid	≠ Dirty	行内容是否可用 vs 是否比下一级更新	miss/写回条件错
Write-through	≠ Write-allocate	hit 更新传播轴 vs miss 是否取块轴	从一个策略推另一个
Hit rate	≠ Total time	命中比例 vs hit/miss 成本综合	只按命中率判快慢
Cache miss	≠ TLB miss	数据副本未命中 vs 翻译副本未命中	把重试处理者写错

10 做题控制协议

1. 统一编址单位、主存地址宽度、数据容量、块大小和相联度；

2. 算 line/set/offset/index/tag，并把访问转为 ‘(block,set,tag,offset)’；

3. 逐次推进 valid/tag/dirty/replacement 状态；

4. miss 写 victim、write-back、fill、策略和 retry；

5. AMAT 先声明成本模型，避免重复计算同一 penalty；

6. 用边界地址、冲突地址和替换后重访独立验证。

压缩成第一动作

看到 Cache 题，先问：index 找到哪些候选？valid+tag 如何证明身份？offset 取哪一段？miss 时谁被替换、谁拥有最新数据？

11 全科坐标与跨册交接

Map Coordinate: Data Location / Fast-Slow Path

CO-06 负责回答“目标块现在在哪个副本、怎样证明身份、miss 后向哪一级请求”。它消费 CO-05 的存储服务模型，向 CO-07/B02 提供物理地址身份接口，并向 CO-04/I01 输出 hit/miss、fill、stall 与数据 ready 时刻。

Cache hit rate 不是完整 Cost；必须同时考虑 hit time、miss penalty、写回/填充流量、替换元数据和与流水线的重叠。它也不等于权限成功，更不等于 OS Page Cache 命中。

12 考纲映射与来源处理

考点	本册机制	接口
局部性/层次	block transfer、hit/miss、AMAT	CO-05
映射与替换	地址三分、3C、victim state	Rules
写策略/容量	dirty、写回、物理位数	CO-05/CO-04
TLB/Cache 组合	PIPT/VIPT/VIVT 最小边界	CO-07/CO-B02

一句话

Cache 的正确性来自“候选位置 + valid/tag 身份证明 + offset 定位 + 写状态管理”，性能才是在这个正确副本系统上由局部性、替换和 AMAT 产生的结果。

本册吸收归档《Cache》《存储器层次》的地址、状态、替换、写策略和性能主干；固定 hit 周期、精确 LRU 位数、所有 miss 都阻塞整机等实现特例保留为题设条件。